=== UNIT 4: MODULAR PROGRAMMING, FUNCTIONS & RECURSION ===

>> 4.1 Function Architecture, Parameters and Return Values

Functions are reusable, named blocks of code designed to perform specific computational tasks, promoting code modularity, abstraction, and maintainability:
* Defining Functions: def function_name(param1, param2=default_val): docstring; body; return result.
* Call-by-Object-Reference (Pass-by-Assignment): Arguments are passed by object reference. If an argument is mutable (e.g. a list), modifications made inside the function directly mutate the caller's object. If immutable, reassignments remain local.
* Positional vs Keyword Arguments: Arguments can be passed positionally or explicitly by name (key=value).
* Arbitrary Argument Lists (*args and **kwargs):
- *args: Packs arbitrary positional arguments into an immutable tuple.
- **kwargs: Packs arbitrary keyword arguments into a dictionary.

>> 4.2 Variable Scope and the LEGB Resolution Rule

Python resolves variable names following the strict hierarchical LEGB Rule:
* 1. L (Local): Names assigned inside the executing function body.
* 2. E (Enclosing): Names defined in the local scope of enclosing/outer functions (closures).
* 3. G (Global): Names defined at top-level module scope or declared via global keyword.
* 4. B (Built-in): Python pre-loaded built-in functions and constants (range, len, print, int).
* The global and nonlocal Keywords: global binds a variable to module-level scope; nonlocal binds a variable to the nearest enclosing non-global scope in nested functions.

>> 4.3 Anonymous Functions (Lambda Expressions) and Functional Tools

* Lambda Functions: Syntactic shortcut for small, single-expression anonymous functions: lambda arg1, arg2: expression.
* Functional Primitives:
- map(func, iterable): Applies func to every element of iterable.

- filter(predicate, iterable): Filters elements satisfying predicate(item) == True.
- functools.reduce(func, iterable): Cumulatively applies binary func to reduce iterable to a single scalar value.

>> 4.4 Recursion: Principle, Call Stack and Base Condition

Recursion is a programming technique where a function calls itself to solve smaller sub-problems of the same structure:
* Essential Components: (1) Base Case: Non-recursive termination condition preventing infinite recursion. (2) Recursive Step: Breaking problem down toward the base case.
* Call Stack Dynamics: Each recursive invocation pushes a new stack frame (containing local variables, parameters, and return address) onto the Call Stack. Stack frames unwind in LIFO order upon reaching the base case.
* Recursion Limit: sys.getrecursionlimit() defaults to 1000 in CPython to prevent stack overflow errors.

[PROGRAM 1: Variable Arguments (*args, **kwargs) Summary Engine]

Problem Context: Write a universal mathematical summary function that accepts arbitrary numeric arguments via *args, optional configuration flags via **kwargs (e.g., round_to, compute_stats), and returns formatted analytical metrics.
Python Source Code:

```
def compute_summary(*args, **kwargs):
if not args:
return {'error': 'No data provided'}

total = sum(args)
mean = total / len(args)
sorted_data = sorted(args)
median = sorted_data[len(args)//2] if len(args) % 2 != 0 else (sorted_data[len(args)//2
- 1] + sorted_data[len(args)//2]) / 2

precision = kwargs.get('round_to', 2)
return {
'count': len(args)
'sum': round(total, precision)
'mean': round(mean, precision)
'median': round(median, precision)
'min': min(args)
```

'max': max(args)
}

stats = compute_summary(45, 88, 12, 90, 67, 34, round_to=3)
for k, v in stats.items():
print(f' {k:8s} : {v}')
Execution Trace & Output:
Input: 6 integers. count: 6, sum: 336, mean: 56.0, median: 56.0, min: 12, max: 90.

[PROGRAM 2: Recursive Binary Search Algorithm]

Problem Context: Implement an efficient recursive binary search function that searches for target key in a sorted list in O(log n) time.
Python Source Code:
def binary_search(arr, low, high, target):
if low > high:
return -1 # Base Case: target not found

mid = (low + high) // 2
if arr[mid] == target:
return mid # Base Case: target found at mid index
elif arr[mid] > target:
return binary_search(arr, low, mid - 1, target) # Search left half
else:
return binary_search(arr, mid + 1, high, target) # Search right half

data = [11, 23, 34, 45, 56, 67, 78, 89, 90, 99]
target = 67
index = binary_search(data, 0, len(data) - 1, target)
print(f'Array: {data}')
print(f'Element {target} found at index: {index}')
Execution Trace & Output:
Call 1: low=0, high=9, mid=4 (arr[4]=56 < 67) -> Search right (low=5, high=9).
Call 2: low=5, high=9, mid=7 (arr[7]=89 > 67) -> Search left (low=5, high=6).
Call 3: low=5, high=6, mid=5 (arr[5]=67 == 67) -> Return index 5.

[PROGRAM 3: Tower of Hanoi Recursive Solver]

Problem Context: Solve the classic Tower of Hanoi problem for N disks moving from source rod A to target rod C using auxiliary rod B. Print every individual move.
Python Source Code:

```
move_count = 0
def tower_of_hanoi(n, source, target, auxiliary):
global move_count
if n == 1:
move_count += 1
print(f'Step {move_count:2d}: Move disk 1 from {source} -> {target}')
return
tower_of_hanoi(n - 1, source, auxiliary, target)
move_count += 1
print(f'Step {move_count:2d}: Move disk {n} from {source} -> {target}')
tower_of_hanoi(n - 1, auxiliary, target, source)

disks = 3
print(f'Solving Tower of Hanoi for {disks} disks (Total moves = 2^{disks} - 1 =
{2**disks - 1}):')
tower_of_hanoi(disks, 'A', 'C', 'B')
```

Execution Trace & Output:
7 moves executed cleanly: Move 1 from A->C, 2 from A->B, 1 from C->B, 3 from A->C, 1 from B->A, 2 from B->C, 1 from A->C.

[PROGRAM 4: Functional Data Pipeline with Map, Filter and Lambda]

Problem Context: Given a list of employee records with salaries, filter employees earning below threshold, apply an 8% inflation adjustment raise, and compute total payroll using map, filter, and lambda.
Python Source Code:

```
from functools import reduce

employees = [
{'name': 'Aarav', 'salary': 45000}
{'name': 'Diya',  'salary': 72000}
{'name': 'Rohan', 'salary': 38000}
{'name': 'Kavita','salary': 85000}
]

# 1. Filter: Employees earning under 70,000
lower_bracket = list(filter(lambda emp: emp['salary'] < 70000, employees))
```

```
# 2. Map: Apply 8% salary adjustment
adjusted = list(map(lambda emp: {**emp, 'salary': round(emp['salary'] * 1.08)},
lower_bracket))

# 3. Reduce: Compute total payroll for adjusted cohort
total_cost = reduce(lambda acc, emp: acc + emp['salary'], adjusted, 0)

print('Adjusted Records:', adjusted)
print(f'Total Adjusted Payroll: INR {total_cost:,}')
```

Execution Trace & Output:
Filtered: Aarav (45k), Rohan (38k). Adjusted: Aarav (48,600), Rohan (41,040). Total payroll: INR 89,640.

>> 4.5 University Review & Examination Questions

* Q1: Explain the LEGB rule for variable scope resolution in Python with an illustrative code example.
Answer: The LEGB rule defines the sequential lookup order Python uses to resolve variable names: (1) Local: inside current function. (2) Enclosing: enclosing function scopes from nearest to outermost. (3) Global: module-level variables. (4) Built-in: Python core built-ins (e.g. abs, len). Example: x = 'Global'; def outer(): x = 'Enclosing'; def inner(): x = 'Local'; print(x) (prints 'Local'). If 'Local' x is commented out, Python checks Enclosing ('Enclosing'), then Global ('Global'), and finally Built-in.
* Q2: Explain the call-by-object-reference mechanism in Python and compare the effects of passing mutable vs immutable objects.
Answer: Python passes arguments using 'call-by-object-reference'. The function parameter receives a copy of the reference (pointer) to the actual object. If the object is IMMUTABLE (int, float, str, tuple), modifying the parameter inside the function rebinds the local variable to a newly created object, leaving the caller's variable unchanged. If the object is MUTABLE (list, dict, set), in-place mutating methods (e.g. list.append() or dict[key]=val) directly alter the underlying heap object, and changes are immediately reflected in the caller's environment.

>> 4.6 Key Takeaways and Revision Points

* *args packs positional arguments into a tuple; **kwargs packs keyword arguments into a dictionary.
* Scope resolution follows LEGB: Local -> Enclosing -> Global -> Built-in.

* Recursive algorithms require a well-defined base case to avoid stack overflow.
* Lambda functions provide concise inline function expressions.